Loi d’induction de Faraday : Le fonctionnement d’une guitare électrique



Problématique

Comment **le type de micro** influence-t-il le **timbre** et **la qualité** du **son** d'une guitare électrique ?

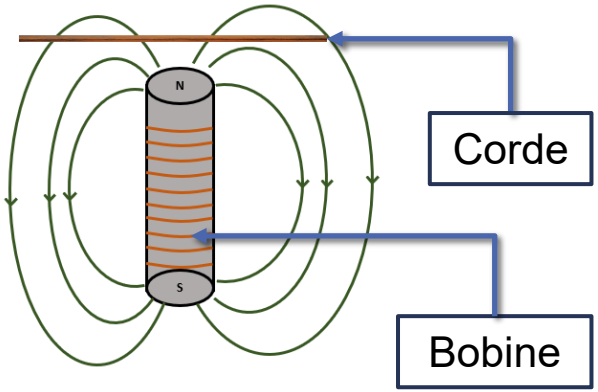
Types de micro :

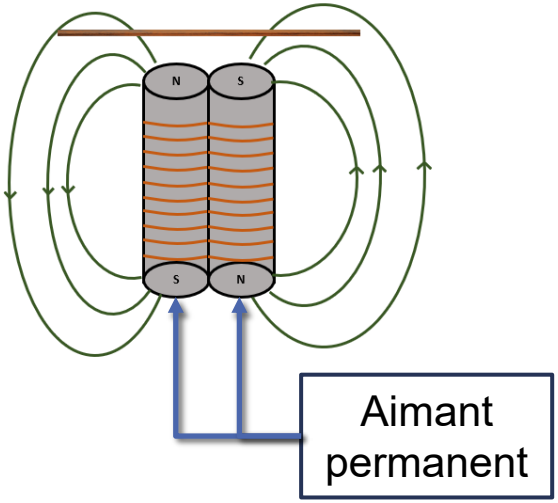
- SingleCoil
(ou simple bobinage)
- Humbucker
(ou double bobinage)

Comment fonctionne une guitare électrique ?



Source image : Ibanez.com





Modélisation : Force électromotrice FEM du micro



Hypothèses :
- Vibration de la corde : Onde stationnaire
- Champs Magnétique de la corde Uniforme

Notations :
- e : F.E.M. dans le micro
- N : Nombre de spire du micro
- S : Surface du micro
- $\frac{dB}{dy}$: Gradient du champ B de la corde
- f_0 : Fréquence fondamentale de la corde
- A : Amplitude initiale de vibration de la corde
- L : Longueur de la corde
- x_0 : Position de la source de la vibration
- x_m : Position du micro

Initialement on excite la corde telle que :



L'équation d'onde de la corde est :

$$y(x,t) = \sum_{n\geq 1} B_n \sin(\frac{n\pi x}{L}) \cos(2\pi f_n t)$$

Avec la **Loi d'induction de faraday** :

$$e(t) = -N\frac{d\phi_B}{dt} = -NS\frac{dB}{dy}\frac{dy(x_m,t)}{dt}$$

D'où

$$e(t) = NS\frac{dB}{dy}\sum_n [A_n B_n \sin(2\pi f_n t)\,]$$

Avec : $f_n = nf_0$ $\quad B_n = \dfrac{2A\sin(\frac{n\pi}{L}x_0)}{\frac{x_0}{L}\left(1-\frac{x_0}{L}\right)\pi^2 n^2}$ $\quad A_n = 2\pi f_n \sin\left(\frac{n\pi}{L}x_m\right)$

Modélisation : Force électromotrice FEM du micro



Hypothèses :
- Vibration de la corde : Onde stationnaire
- Champs Magnétique de la corde Uniforme

Notations :
- e : F.E.M. dans le micro
- N : Nombre de spire du micro
- S : Surface du micro
- $\frac{dB}{dy}$: Gradient du champ B de la corde
- f_0 : Fréquence fondamentale de la corde
- A : Amplitude initiale de vibration de la corde
- L : Longueur de la corde
- x_0 : Position de la source de la vibration
- x_1 : Position du 1er bobinage
- x_2 : Position du 2e bobinage

Initialement on excite la corde telle que :



L'équation d'onde de la corde est :

$$y(x,t) = \sum_{n\geq 1} B_n \sin(\frac{n\pi x}{L}) \cos(2\pi f_n t)$$

Avec la **Loi d'induction de faraday** :

$$e(t) = -N\frac{d\phi_B}{dt} = -NS\frac{dB}{dy}(\frac{dy(x_1,t)}{dt} + \frac{dy(x_2,t)}{dt})$$

D'où

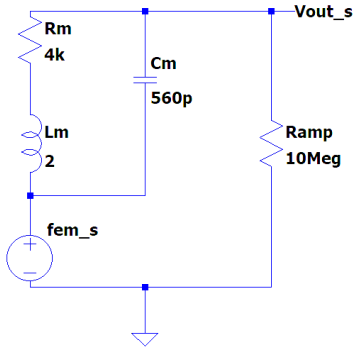
$$e(t) = NS\frac{dB}{dy}\sum_n [A_n B_n \sin(2\pi f_n t)\,]$$

Avec : $f_n = nf_0$ $\quad B_n = \frac{2A\sin(\frac{n\pi}{L}x_0)}{\frac{x_0}{L}\left(1-\frac{x_0}{L}\right)\pi^2 n^2}$ $\quad A_n = 2\pi f_n(\sin\left(\frac{n\pi}{L}x_1\right) + \sin(\frac{n\pi}{L}x_2))$

Modélisation : Impédance et réponse fréquentielle

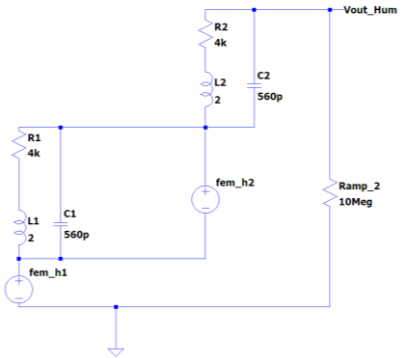


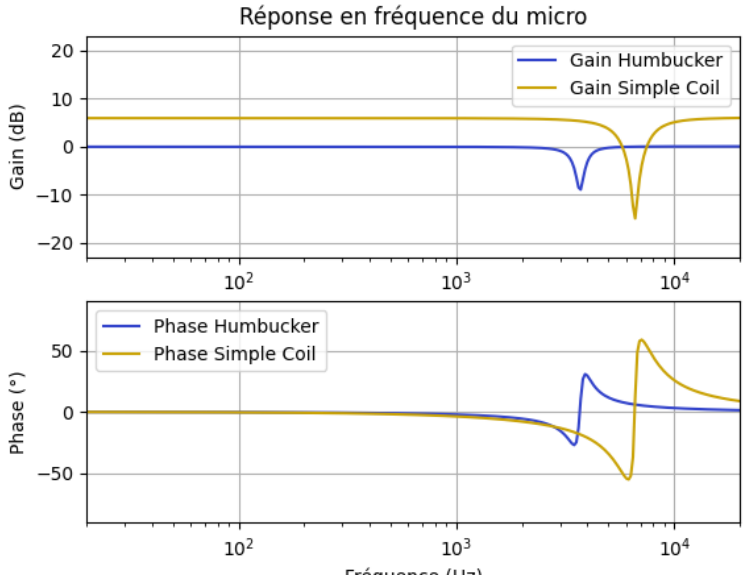
Single-Coil Pickup





Humbucker





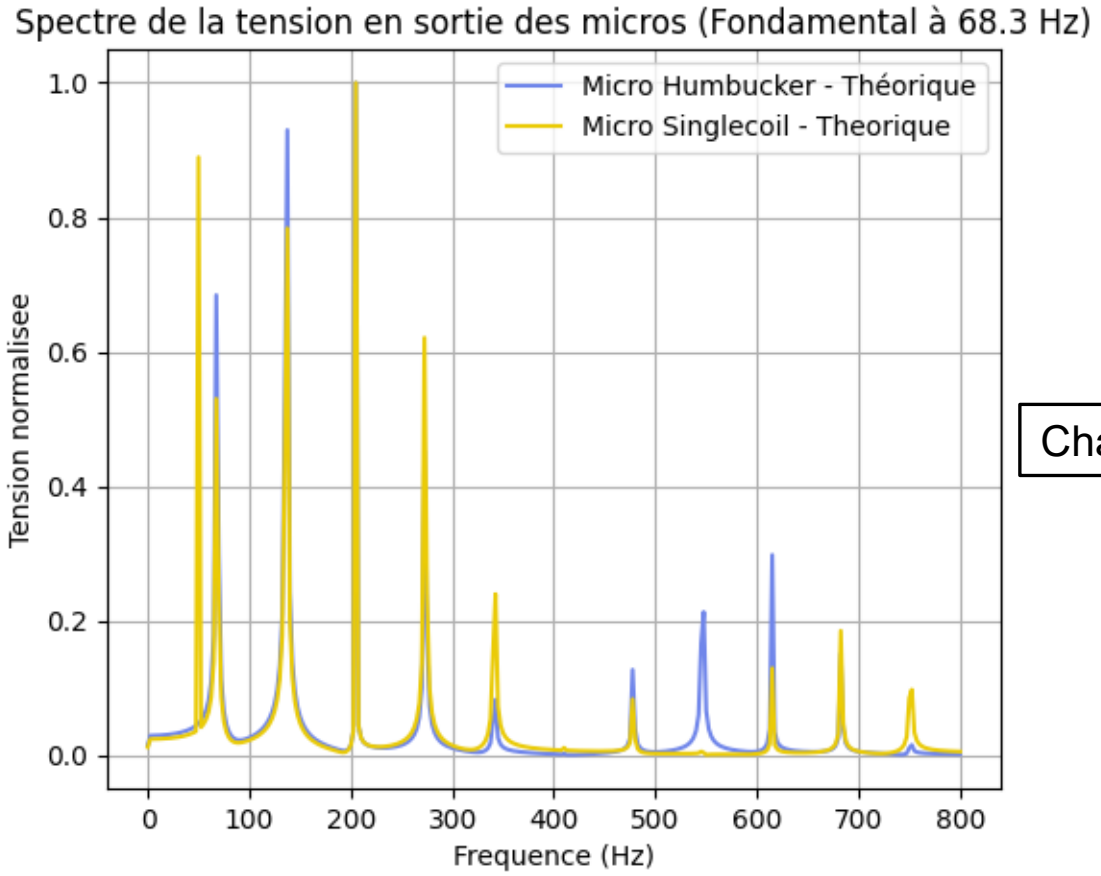
On observe : $f_0 = \frac{1}{2\pi\sqrt{L_m C_m}} > 3000$ Hz

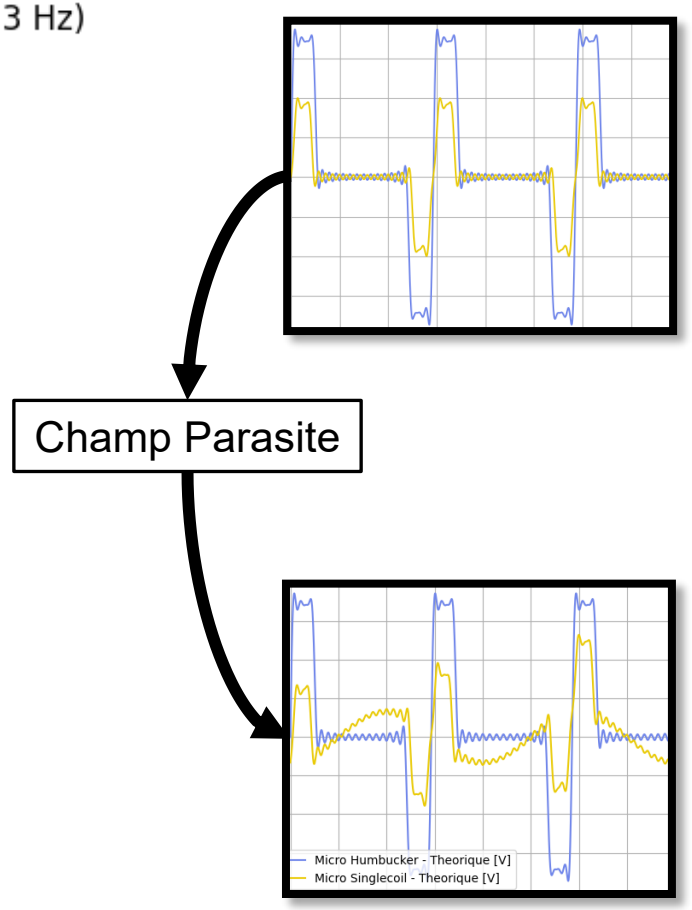
Ainsi on fera l'hypothèse : $\boxed{e(t) = s(t)}$

Notations :

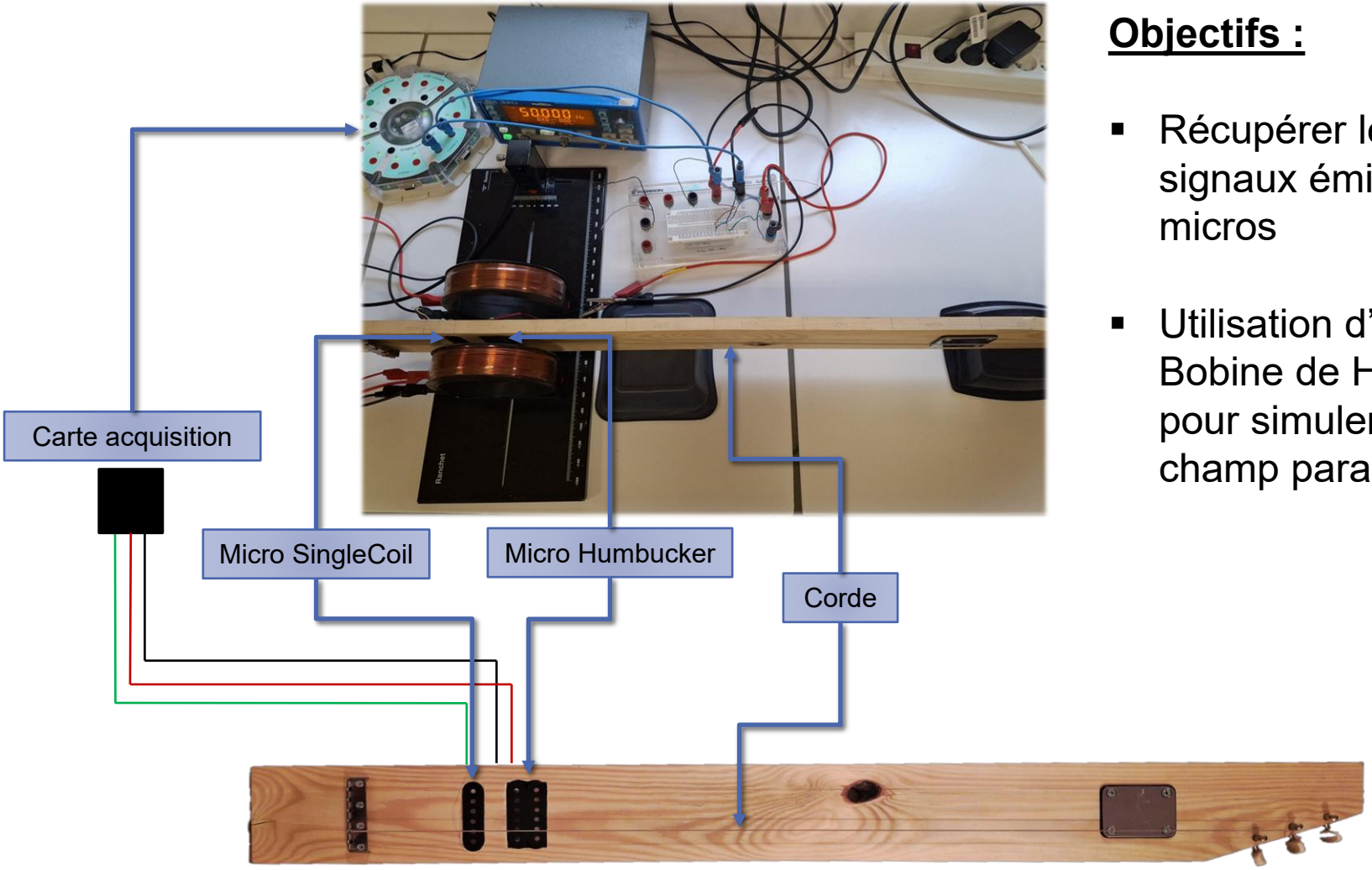
- $e(t)$: FEM totale dans le micro
- $s(t)$: Tension de sortie du micro

Modélisation : Conclusion





Expérimentation : Montage



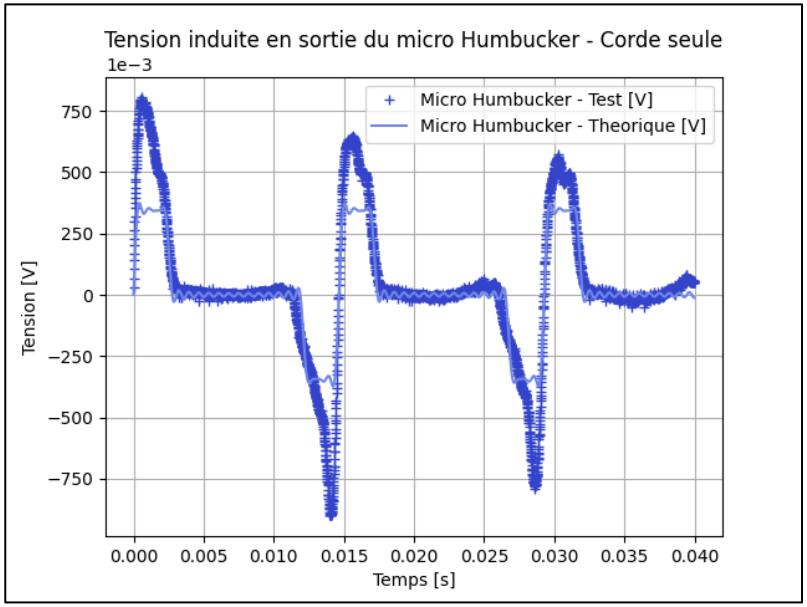
Objectifs :

- Récupérer les signaux émis par les micros
- Utilisation d'une Bobine de Helmholtz pour simuler le champ parasite

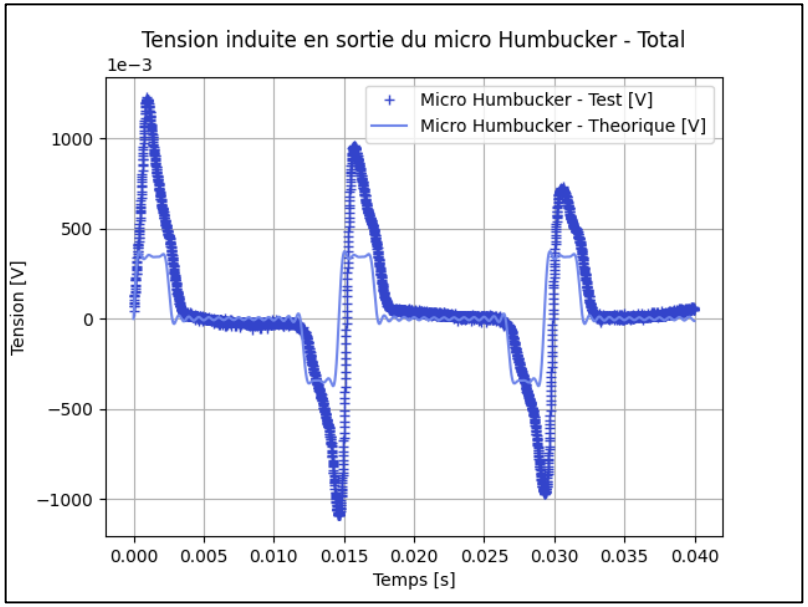
Expérimentation : Résultats

	SingleCoil	Humbucker
Corde seule		✓
Corde + Parasite		✓

✓	▪ Même Période ▪ Même Harmonique
✗	▪ Amplitudes différentes → Car Amplitude initiale différentes



Sans Champs Parasite



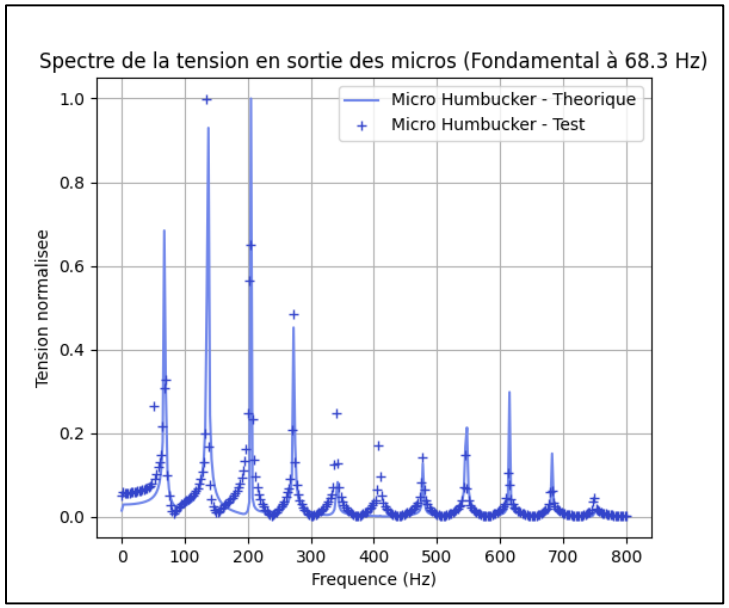
Avec Champ Parasite

- Comparer avec Micro Simple

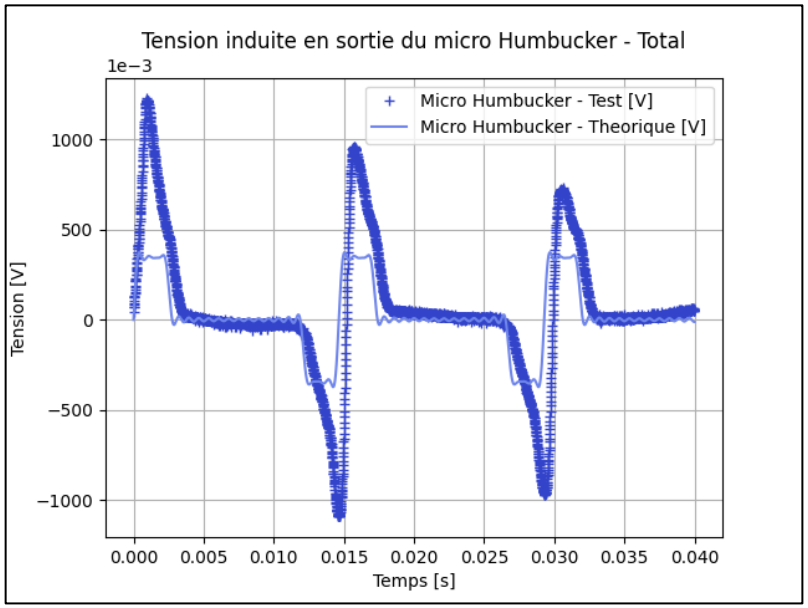
Expérimentation : Résultats

	SingleCoil	Humbucker
Corde seule		✓
Corde + Parasite		✓

✓	▪ Même Période ▪ Même Harmonique
✗	▪ Amplitudes différentes → Car Amplitude initiale différentes



Spectre FFT



Avec Champ Parasite

▪ Comparer avec Micro Simple

Conclusion

Jusque maintenant…

- Modèle cohérent
- Expérience cohérente

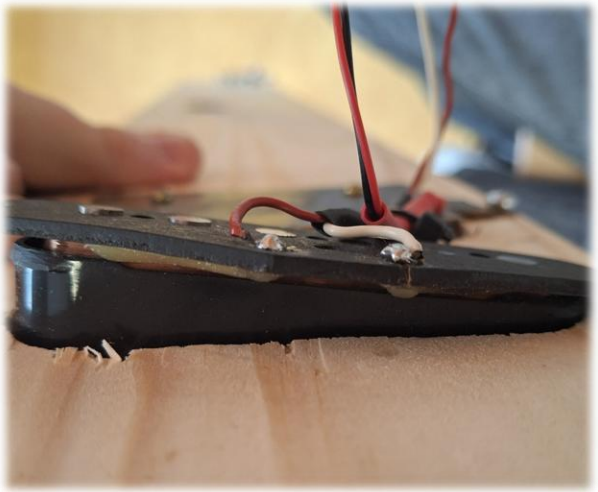
	SingleCoil	Humbucker
Corde seule	…	✓
Corde + Parasite	…	✓

Et pour la suite ?

- Expérimenter le micro Singlecoil
- Tracer la droite $e(t) = f(B_0)$

Mais…

- Pas de signal pour le micro Singlecoil



Annexes : Sommaire

1. Sources et Bibliographie
2. Remerciements
3. Loi d'induction de Faraday
4. Démonstration du modèle Théorique
    1. Micro SingleCoil excité par une corde
    2. Micro Humbucker excité par une corde
    3. Micro SingleCoil & Humbucker excité par un Champ Parasite
5. Expérimentation
    1. Dimensionnement
6. Python
    1. Réponse fréquentielle du micro
    2. Modèle Théorique & Résultats
        1. Initialisation du programme
        2. Fonctions du modèle théorique
        3. Fonctions utiles
        4. Calcul du modèle théorique
        5. Calcul et affichage des FFT
        6. Enregistrement des courbes

1. Sources & Bibliographie

[1] Matthew Hill. "The hidden history of the electric guitar." Benoît Navarret, Marc Battier, Philippe Bruguière, Philippe Gonin. ''Quand la guitare [s']électrise!'', Sorbonne Université Presses, pp.33-61, 2022.

[2] Rickenbacker, "Rickenbacker Legacy", Rickenbacker,
[En ligne]. Disponible sur : https://www.rickenbacker.com/legacy/

[3] S. W. Duncan, "Interview With Seth Lover," Seymour Duncan,
[En ligne]. Disponible sur : https://www.seymourduncan.com/blog/latest-updates/seymour-w-duncans-interview-with-seth-lover

[4] Larry, ''How to prevent humming and noise in a guitar amp'', Thomann,
[En ligne]. Disponible sur : https://www.thomann.de/blog/en/learn/how-to-prevent-humming-and-noise-in-a-guitar-amp/

[5] Dan Carson. '' Investigation of the Electromagnetic Properties of Electric Guitar Pickups". University of Illinois at Urbana-Champaign. USA. 2009

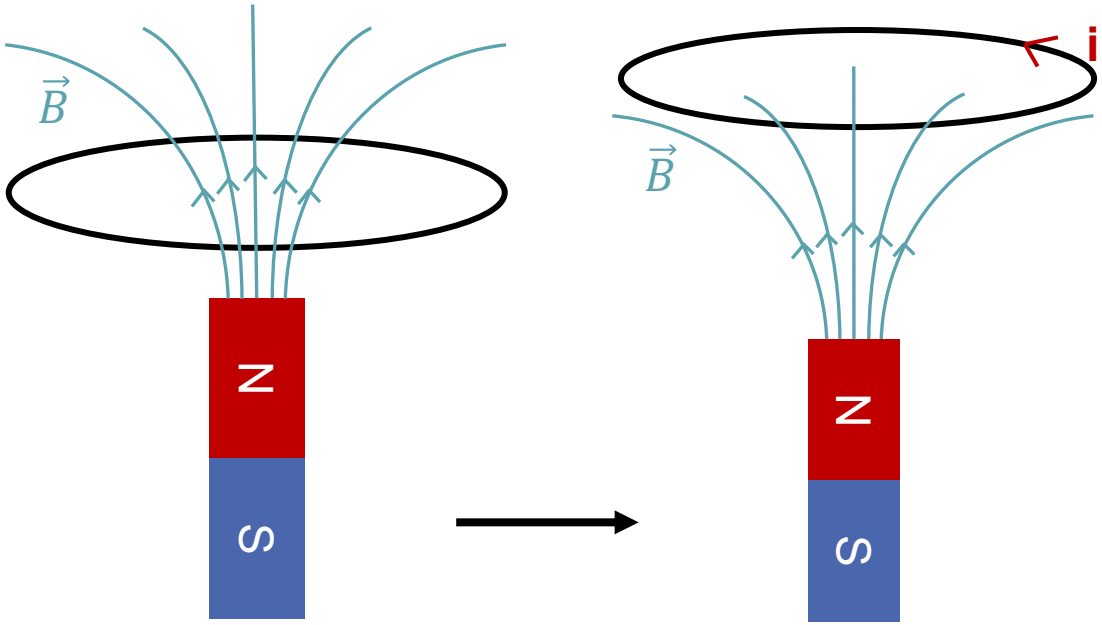
[6] P. Perov, W. Johnson, and N. Perova-Mello, "The physics of guitar string vibrations," *American Journal of Physics*, vol. 84, no. 1, pp. 38–43, Jan. 2016.

2. Remerciements

- **Art Music Store**, 3800, Grenoble, Place Victor Hugo

- **Menuiserie Paradis**, 38590, Brézins, Rue des Caillères

3. Loi d'induction de Faraday



Dans un circuit fermé conducteur, une variation de flux magnétique entraine la formation d'un courant induit

L'effet de la F.E.M induite est tel qu'il s'oppose à la variation de flux

$$e = -N\frac{d\phi_B}{dt}$$

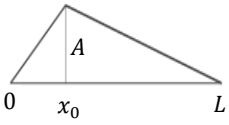
4.1 Modèle Théorique : Micro Simple *cf. [1]*

L'étude de la corde est basée sur l'équation des ondes, où la vitesse de propagation v est défini par la tension T et la masse linéique μ

$$\frac{\partial^2 y}{\partial x^2} = \frac{1}{v^2}\frac{\partial^2 y}{\partial t^2}, \text{ avec } v = \sqrt{T/\mu}$$

Pour une corde de longueur L, tiré à la position x_0 d'une distance $y = A$ on a :

$$y(x,0) = \begin{cases} \dfrac{A}{x_0}x, & 0 \le x \le x_0 \\ \dfrac{A}{L-x_0}(L-x), & x_0 \le x \le L \end{cases}$$



Avec des conditions initiales telle que $y(x,0)$ soit la forme initiale de la corde, on a alors :

$$y(x,t) = \sum_{n\geq 1} B_n \sin\frac{n\pi x}{L}\cos\frac{n\pi v t}{L}$$

Avec B_n l'amplitude de l'harmonique n-ème et peut être calculer par :

$$B_n = \frac{2}{L}\int_0^L y(x,0)\sin\frac{n\pi x}{L}dx$$

Ainsi, on a :

$$B_n = \frac{2A}{Lx_0}\int_0^{x_0} x\sin\frac{n\pi x}{L}dx + \frac{2A}{L(L-x_0)}\int_{x_0}^{L}(L-x)\sin\frac{n\pi x}{L}dx$$

D'où :

$$Bn = \frac{2A\sin(\frac{n\pi x_0}{L})}{\frac{x_0}{L}\left(1-\frac{x_0}{L}\right)\pi^2 n^2}$$

La FEM induite $e(t)$ dans la bobine composée de N spire est régie par la Loi d'induction de Faraday :

$$e(t) = -N\frac{d\phi_B}{dt}$$

En supposant que la variation de flux est principalement due au déplacement de la corde :

$$e(t) = -NS\frac{dB}{dy}\frac{dy(x_m,t)}{dt}$$

D'où la relation :

$$e(t) = -NS\frac{dB}{dy}\sum_n[-B_n\sin(\frac{n\pi}{L}x_m)\sin(\frac{n\pi\sqrt{T}}{L\sqrt{\mu}}t)\frac{n\pi\sqrt{T}}{L\sqrt{\mu}}]$$

On pose alors :

$$f_n = \frac{n}{2L}\sqrt{\frac{T}{\mu}} \quad \text{et} \quad A_n = 2\pi f_n \sin\left(\frac{n\pi}{L}x_m\right)$$

Ainsi :

$$e(t) = NS\frac{dB}{dy}\sum_n[A_n B_n\sin(2\pi f_n t)\,]$$

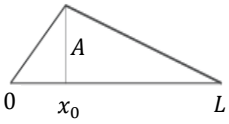
4.2 Modèle Théorique : Micro Humbucker *cf. [1]*

L'étude de la corde est basée sur l'équation des ondes, où la vitesse de propagation v est défini par la tension T et la masse linéique μ

$$\frac{\partial^2 y}{\partial x^2} = \frac{1}{v^2}\frac{\partial^2 y}{\partial t^2}, \text{ avec } v = \sqrt{T/\mu}$$

Pour une corde de longueur L, tiré à la position x_0 d'une distance $y = A$ on a :

$$y(x,0) = \begin{cases} \dfrac{A}{x_0}x, & 0 \le x \le x_0 \\ \dfrac{A}{L-x_0}(L-x), & x_0 \le x \le L \end{cases}$$



Avec des conditions initiales telle que $y(x,0)$ soit la forme initiale de la corde, on a alors :

$$y(x,t) = \sum_{n\ge1} B_n \sin\frac{n\pi x}{L}\cos\frac{n\pi v t}{L}$$

Avec B_n l'amplitude de l'harmonique n-ème et peut être calculer par :

$$B_n = \frac{2}{L}\int_0^L y(x,0)\sin\frac{n\pi x}{L}dx$$

Ainsi, on a :

$$B_n = \frac{2A}{Lx_0}\int_0^{x_0} x\sin\frac{n\pi x}{L}dx + \frac{2A}{L(L-x_0)}\int_{x_0}^{L}(L-x)\sin\frac{n\pi x}{L}dx$$

D'où :

$$Bn = \frac{2A\sin(\frac{n\pi x_0}{L})}{\frac{x_0}{L}\left(1-\frac{x_0}{L}\right)\pi^2 n^2}$$

La FEM induite $e(t)$ dans la bobine composée de N spire est régie par la Loi d'induction de Faraday :

$$e(t) = -N\frac{d\phi_B}{dt}$$

En supposant que la variation de flux est principalement due au déplacement de la corde :

$$e(t) = -NS\frac{dB}{dy}\left(\frac{dy(x_1,t)}{dt} + \frac{dy(x_2,t)}{dt}\right)$$

D'où la relation :

$$e(t) = -NS\frac{dB}{dy}\sum_n\left[-B_n\left(\sin\left(\frac{n\pi}{L}x_1\right) + \sin\left(\frac{n\pi}{L}x_2\right)\right)\sin\left(\frac{n\pi\sqrt{T}}{L\sqrt{\mu}}t\right)\frac{n\pi\sqrt{T}}{L\sqrt{\mu}}\right]$$

On pose alors :

$$f_n = \frac{n}{2L}\sqrt{\frac{T}{\mu}} \quad \text{et} \quad A_n = 2\pi f_n\left(\sin\left(\frac{n\pi}{L}x_1\right) + \sin\left(\frac{n\pi}{L}x_2\right)\right)$$

Ainsi :

$$e(t) = NS\frac{dB}{dy}\sum_n\left[A_n B_n \sin(2\pi f_n t)\right]$$

4.3 Modèle Théorique : Action d'un champ parasite

On étudie un champ parasite d'amplitude B0, de fréquence f et de gradient alpha régie par l'équation:

$$B(x,t) = B_0 \sin(2\pi f_B t)$$

Ainsi on a :

$$\frac{\partial B(x,t)}{\partial t} = B_0 2\pi f_B \cos(2\pi f_B t)$$

La FEM induite $e(t)$ dans la bobine composée de N spire est régie par la Loi d'induction de Faraday :

$$e(t) = -N\frac{d\phi_B}{dt}$$

Pour un micro SingleCoil :
En supposant que la variation de flux est principalement due au champ parasite :

$$e(t) = -NS\frac{\partial B(x_m,t)}{\partial t}$$

Ainsi :

$$e(t) = -NSB_0 2\pi f_B \cos(2\pi f_B t)$$

Pour un micro Humbucker :
Par Linéarité de la dérivée, on a alors :

$$e(t) = e_1(t) + e_2(t)$$

Le micro est composé de deux bobine à N spire câblée en opposition de phase :

$$e(t) = -NS\frac{\partial B(x_1,t)}{\partial t} + \left(-\left(-NS\frac{\partial B(x_1,t)}{\partial t}\right)\right)$$

C'est-à-dire :

$$e(t) = -NS\left(\frac{\partial B(x_1,t)}{\partial t} - \frac{\partial B(x_2,t)}{\partial t}\right)$$

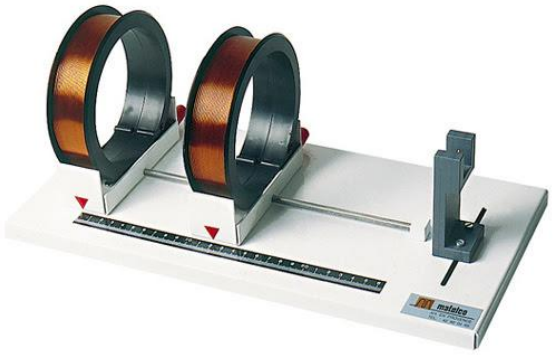
D'où :

$$e(t) = -NSB_0 2\pi f_B \cos(2\pi f_B t)\,|x_1 - x_2|$$

NB :

- On a une annulation totale du bruit si et seulement si x_1 et x_2 proche
- En pratique $|x_1 - x_2| = 0.02$ mètre
- D'où l'efficacité du Humbucker

5.1 Dimensionnement de l'expérimentation



Comment générer et contrôler un champ magnétique parasite ?

Notations :

- U_0 : Amplitude du signal électrique généré par le GBF
- f : Fréquence du signal électrique généré par le GBF
- n : Nombre de spire par bobine
- L_B : Inductance de chaque bobine
- R_B : Resistance de chaque bobine
- R : Rayon de chaque bobine
- μ_0 : Perméabilité du vide
- $B(t)$: Champ magnétique au centre du système

Le GBF délivre une tension :

$$u(t) = U_0 \cos(2\pi f_B t)$$

Ainsi l'intensité dans la bobine de Helmholtz est :

$$i(t) = \frac{U_0}{\sqrt{R_B^2 + (2\pi f L_B)^2}} \cos(2\pi f t)$$

Ainsi on obtient la relation :

$$\mathrm{B}(t) = \left(\frac{4}{5}\right)^{3/2} \frac{\mu_0 n}{R} \frac{U_0}{\sqrt{R_B^2 + (2\pi f L_B)^2}} \cos(2\pi f t)$$

☛ Ordre de Grandeur : $B_0 = 100\ \mu T$

6.1 Réponse fréquentielle du micro

```python
#!#####################!#
#!   Bibliothèques    !#
#!#####################!#

print("\nImportation des bibliotheques...")
import numpy as np
import matplotlib.pyplot as plt
print("Bibliotheques importees.")

#!#####################!#
#!     Variables      !#
#!#####################!#

print("\nInitialisation des variables...")
freq = []
Vvout_hum_db = []
Vvout_hum_deg = []
Vvout_sin_db = []
Vvout_sin_deg = []

Color_c = "#505050"
Color_h = "#3344CB"
Color_s = "#CAA307"

print("Variables initialisees.")

#!#####################!#
#! Lectures Fichiers  !#
#!#####################!#

print("\nLecture du fichier Bode.txt...")

file = open("Python/Ressource Model/Bode.txt")
L_lignes = file.readlines()

for i in range(1, len(L_lignes)):
    values = L_lignes[i].strip().split("\t")
    values = [float(v.replace("\n", "")) for v in values]
    freq.append(values[0])
    Vvout_hum_db.append(values[1])
    Vvout_hum_deg.append(values[2])
    Vvout_sin_db.append(values[3])
    Vvout_sin_deg.append(values[4])

file.close()
print("Fichier Bode.txt lu.")
print("Variables remplies.")
```

```python
#!#####################!#
#!     Affichage      !#
#!#####################!#

print("\nAffichage du diagramme de Bode...")
plt.figure(1)

plt.subplot(2, 1, 1)
plt.semilogx(freq, Vvout_hum_db, color=Color_h, label="Gain Humbucker")
plt.semilogx(freq, Vvout_sin_db, color=Color_s, label="Gain Simple Coil")
plt.grid()
plt.ylabel("Gain (dB)")
plt.ylim(-3, 23)
plt.xlim(20, 20000)
plt.legend()
plt.title("Réponse en fréquence du micro")

plt.subplot(2, 1, 2)
plt.semilogx(freq, Vvout_hum_deg, color=Color_h, label="Phase Humbucker")
plt.semilogx(freq, Vvout_sin_deg, color=Color_s, label="Phase Simple Coil")
plt.grid()
plt.ylabel("Phase (°)")
plt.ylim(-45, 15)
plt.xlim(20, 20000)
plt.xlabel("Fréquence (Hz)")
plt.legend()

plt.savefig("Python/Exports/Bode.png")
plt.close()
print("Diagramme de Bode sauvegarde sous Python/Exports/Bode.png.")

print("\n---\nFin de l'execution de Modele_elec.py\n---\n")
```



Données récoltées avec LTSpice

6.2.1 Initialisation du programme



```python
#! ------------------ Constantes --------------------------------
print("-" * 100)
print("\nInitialisation des constantes")
#* Constante physique
rho = 1.68 * 10**(-8)
mu0 = 4e-7 * np.pi
#* Parametre maquette
xm = 0.105
x1 = 0.140
x2 = 0.160
L = 0.845
mu = 9 * 10**(-3)
dBdy = 0.05             #A verifier ???
#* Parametre Micros
d = 6.3 * 10**(-5)
S = 7 * 10**(-4)
#* Parametre Micro Singlecoil
RCC_s = 8.92 * 10**3      #A verifier ???
lmoy_s = 0.147
N_s = int(RCC_s * np.pi * d**2 / (4 * rho * lmoy_s))
N_s = 8000
#* Parametre Micro Humbucker
RCC_h = 4.52 * 10**3
lmoy_h = lmoy_s            #A verifier ???
N_h = int(RCC_h * np.pi * d**2 / (8 * rho * lmoy_h))
#* Parametre Champ parasite
Nbob = 95
R = 0.130 / 2
Lbob = 1                #A verifier ???
Rbob = 10               #A verifier ???
fB = 50
U0 = 10
I0 = 7                  #A verifier ???
#* Parametre variable
A = 35 * 10**(-4)          #A verifier ???
x0 = 0.140                  #A verifier ???
F = 68.3                    #A verifier ???
T = 4 * mu * (L * F)**2
#* Couleurs graphiques
Color_c = "#505050"
Color_h_test = "#3344CB"
Color_s_test = "#CAA307"
Color_h = "#7086EA"
Color_s = "#EACB07"
```

```python
#! ------------------ Bibliotheques ------------------------------
print("-" * 100)
print("\nImportation des bibliotheques...")
import os, logging, numpy as np, matplotlib.pyplot as plt
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')
print("Bibliotheques importees.")

```

6.2.2 Fonctions du modèle théorique

```python
def entree(n: int, x: float, t: np.ndarray) -> np.ndarray:
    s = 0
    v = np.sqrt(T / mu)
    for i in range(1, n + 1):
        Bn = (2 * A * np.sin((i * np.pi * x0) / (L))) / ((x0) / (L) * (1 - (x0) / (L)) * np.pi**2 * i**2)
        s += Bn * np.sin(i * np.pi * x / L) * np.cos(i * np.pi * v * t / L)
    return s


def fem_singlecoil(n: int, N: int, xm: float, t: np.ndarray) -> np.ndarray:
    s = 0
    for i in range(1, n + 1):
        fn = (i / (2 * L)) * np.sqrt(T / mu)
        An = 2 * np.pi * fn * np.sin(i * np.pi * xm / L)
        Bn = (2 * A * np.sin((i * np.pi * x0) / (L))) / ((x0) / (L) * (1 - (x0) / (L)) * np.pi**2 * i**2)
        s += An * Bn * np.sin(2 * np.pi * fn * t)
    return N * S * dBdy * s


def fem_humbucker(n: int, N: int, x1: float, x2: float, t: np.ndarray) -> np.ndarray:
    s = 0
    for i in range(1, n + 1):
        fn = (i / (2 * L)) * np.sqrt(T / mu)
        An = 2 * np.pi * fn * (np.sin(i * np.pi * x1 / L) + np.sin(i * np.pi * x2 / L))
        Bn = (2 * A * np.sin((i * np.pi * x0) / (L))) / ((x0) / (L) * (1 - (x0) / (L)) * np.pi**2 * i**2)
        s += An * Bn * np.sin(2 * np.pi * fn * t)
    return N * S * dBdy * s


def B(x: float, f: float, t: np.ndarray) -> np.ndarray:
    return (Nbob * mu0 * Rbob**2) / 2 * (1 / (Rbob**2 + (x - Rbob / 2)**2)**(3 / 2) + 1 / (Rbob**2 + (x + Rbob / 2)**2)**(3 / 2)) * I0 * np.sin(2 * np.pi * f * t)


def dBdt(x: float, f: float, t: np.ndarray) -> np.ndarray:
    return (Nbob * mu0 * Rbob**2) / 2 * (1 / (Rbob**2 + (x - Rbob / 2)**2)**(3 / 2) + 1 / (Rbob**2 + (x + Rbob / 2)**2)**(3 / 2)) * 2 * I0 * np.pi * f * np.cos(2 * np.pi * f * t)


def fem_par_s(N: int, xm: float, f: float, t: np.ndarray) -> np.ndarray:
    return -N * S * dBdt(0, f, t)


def fem_par_h(N: int, x1: float, x2: float, f: float, t: np.ndarray) -> np.ndarray:
    return -N * S * (dBdt(0, f, t) - dBdt(abs(x2 - x1), f, t))
```

$$y(x,t) = \sum_{n\geq 1} B_n \sin(\frac{n\pi x}{L}) \cos(2\pi f_n t)$$

$$e(t) = NS\frac{dB}{dy}\sum_{n}[A_n B_n \sin(2\pi f_n t)\,]$$

$$B(t) = \frac{\mu_0 n r^2}{2}\left(\frac{1}{r^2 + \left(x - \frac{r}{2}\right)^{\frac{3}{2}}} + \frac{1}{r^2 + \left(x + \frac{r}{2}\right)^{\frac{3}{2}}}\right) i(t)$$

6.2.3 Fonctions utiles

```python
def read_test_file_partial(path: str, line_start: int, line_end: int, keep_all=False):
    """Version qui lit et n'enlève pas la ligne 0 si besoin (utilisé pour petits fichiers)."""
    with open(path, 'r') as doc:
        L_lignes = doc.readlines()
    times = []
    vals = []
    for i in range(line_start, line_end):
        values = L_lignes[i].strip().split("\t")
        values = [float(v.replace("\n", "")) for v in values]
        times.append(values[0])
        vals.append(values[1])
    times = np.array(times)
    times -= times[0]
    return times, np.array(vals)


def save_plot(fig_path: str, *plot_calls, title=None, xlabel=None, ylabel=None, ylim=None, legend=True):
    """Utility to plot provided plot calls (each is a tuple: (x,y,kwargs_or_label))"""
    plt.figure(figsize=(6, 5))
    for call in plot_calls:
        x, y, kwargs = call
        plt.plot(x, y, **kwargs)
    if title:
        plt.title(title)
    if xlabel:
        plt.xlabel(xlabel)
    if ylabel:
        plt.ylabel(ylabel)
    if ylim is not None:
        plt.ylim(ylim)
    plt.grid()
    if legend:
        plt.legend()
    plt.savefig(fig_path)
    plt.close()


def compute_and_normalize_fft(signal: np.ndarray, Ne_fft: int):
    fft = np.fft.fft(signal)
    amp = 2 * np.abs(fft) / Ne_fft
    amp[0] /= 2
    return amp
```

- Lire les fichiers de données
- Enregistrer des courbes
- Normaliser les spectres

6.2.4 Calcul du modèle théorique

```python
print("\nCalcul des tensions induites...")
    print("\nEchantillonnage...")

    Na = 20
    Te = 4000 * 0.00001
    Ne = 4000
    fs = Ne / Te
    Te_fft = 1
    Ne_fft = 100_000
    fs_fft = Ne_fft / Te_fft

    # Affichage des parametres d'echantillonnage
    print("\nParametres d'echantillonnage (avec unites) :")
    print(f"Te = {Te} s")
    print(f"Ne = {Ne} echantillons")
    print(f"fs = {fs:.6g} Hz")
    print(f"Na = {Na}")

    print("\nCalcul des tensions en reponse a la vibration de la corde...")
    temps = np.linspace(0, Te, Ne, endpoint=False)
    corde = entree(Na, xm, temps)
    fem_s = fem_singlecoil(Na, N_s, xm, temps)
    fem_h = fem_humbucker(Na, N_h, x1, x2, temps)

    print("\nCalcul des tensions en reponse au champ parasite...")
    champ_par = B(0, fB, temps)
    fem_s_par = fem_par_s(N_s, xm, fB, temps)
    fem_h_par = fem_par_h(N_h, x1, x2, fB, temps)

    print("\nCalcul des tensions totales...")
    fem_s_tot = fem_s + fem_s_par
    fem_h_tot = fem_h + fem_h_par
```

6.2.5 Calcul et affichages des FFT

```python
if tracer_fft:
    print("-" * 100)
    print("\nCalcul des FFT...")
    print("\nParametres d'echantillonnage (avec unites) :")
    print(f"Te = {Te_fft} s")
    print(f"Ne = {Ne_fft} echantillons")
    print(f"fs = {fs_fft:.6g} Hz")
    print(f"Na = {Na}")

    print("\nCalcul des tensions en reponse a la vibration de la corde...")
    temps_fft = Test_temps_h_fft
    corde_fft = entree(Na, xm, Test_temps_h_fft)
    fem_s_fft = fem_singlecoil(Na, N_s, xm, Test_temps_h_fft)
    fem_h_fft = fem_humbucker(Na, N_h, x1, x2, Test_temps_h_fft)

    print("\nCalcul des tensions en reponse au champ parasite...")
    champ_par_fft = B(0, fB, temps_fft)
    fem_s_par_fft = fem_par_s(N_s, xm, fB, temps_fft)
    fem_h_par_fft = fem_par_h(N_s, x1, x2, fB, temps_fft)

    print("\nCalcul des tensions totales...")
    fem_s_tot_fft = fem_s_fft + fem_s_par_fft
    fem_h_tot_fft = fem_h_fft + fem_h_par_fft

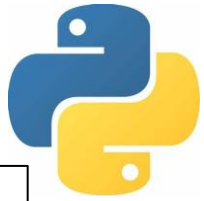
    fft_s_amp = compute_and_normalize_fft(fem_s_tot_fft, Ne_fft)
    fft_h_amp = compute_and_normalize_fft(fem_h_tot_fft, Ne_fft)
    fft_h_test_amp = compute_and_normalize_fft(Test_h_tot_fft, Ne_fft)

    freqs = np.fft.fftfreq(40000, 1 / fs_fft)
    mask = 0 <= freqs
    freqs = freqs[mask]
    mask2 = freqs <= 800
    freqs = freqs[mask2]

    fft_s_amp = fft_s_amp[mask][mask2]
    fft_h_amp = fft_h_amp[mask][mask2]
    fft_h_test_amp = fft_h_test_amp[mask][mask2]

    max_amp_h, max_amp_s, max_amp_test_h = np.max(fft_h_amp), np.max(fft_s_amp), np.max(fft_h_test_amp)
    fft_h_amp = fft_h_amp / max_amp_h
    fft_s_amp = fft_s_amp / max_amp_s
    fft_h_test_amp = fft_h_test_amp / max_amp_test_h
```

6.2.6 Enregistrements des courbes



```python
#* --- Micro Humbucker (Parasite) ---*#
print("Micro Humbucker (Parasite)...")
save_plot("Python/Exports/02-Humbucker_parasite.png",
        (temps, Test_h_p, {'marker': '+', 'linestyle': 'None', 'color': Color_h_test, 'label': "Micro Humbucker - Test [V]"}),
        (temps, fem_h_par, {'color': Color_h, 'label': "Micro Humbucker - Theorique [V]"}),
        title="Tension induite en sortie des micros - Parasite seul",
        xlabel="Temps [s]", ylabel="Tension [V]", ylim=(-0.5, 0.5))
print("Courbes sauvegardees sous Python/Exports/02-Humbucker_parasite.png.")

#* --- Humbucker (total) ---*#
print("Micro Humbucker (total)...")
save_plot("Python/Exports/03-Humbucker_total.png",
        (temps, Test_h_tot, {'marker': '+', 'linestyle': 'None', 'color': Color_h_test, 'label': "Micro Humbucker - Test [V]"}),
        (temps, fem_h_tot, {'color': Color_h, 'label': "Micro Humbucker - Theorique [V]"}),
        title="Tension induite en sortie du micro Humbucker - Total",
        xlabel="Temps [s]", ylabel="Tension [V]")
print("Courbes sauvegardees sous Python/Exports/03-Humbucker_total.png.")

#* --- Total ---*#
print("Tous micros total...")
save_plot("Python/Exports/99-total.png",
        (temps, Test_h_tot, {'marker': '+', 'linestyle': 'None', 'color': Color_h_test, 'label': "Micro Humbucker - Test [V]"}),
        (temps, fem_h_tot, {'color': Color_h, 'label': "Micro Humbucker - Theorique [V]"}),
        (temps, fem_s_tot, {'color': Color_s, 'label': "Micro Singlecoil - Theorique [V]"}),
        title="Tension induite en sortie des micros",
        xlabel="Temps [s]", ylabel="Tension [V]")
print("Courbes sauvegardees sous Python/Exports/99-total.png.")

```